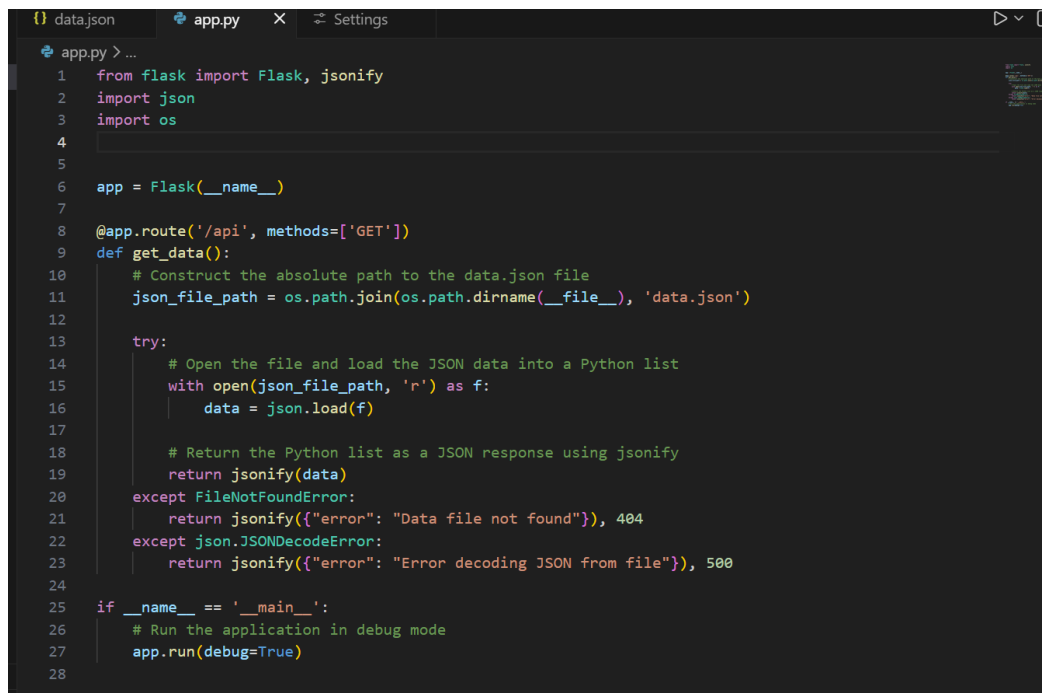


Assignment 3

1. Create a Flask application with an /api route. When this route is accessed, it should return a JSON list. The data should be stored in a backend file, read from it, and sent as a response.

App.py



```
1 from flask import Flask, jsonify
2 import json
3 import os
4
5
6 app = Flask(__name__)
7
8 @app.route('/api', methods=['GET'])
9 def get_data():
10     # Construct the absolute path to the data.json file
11     json_file_path = os.path.join(os.path.dirname(__file__), 'data.json')
12
13     try:
14         # Open the file and load the JSON data into a Python list
15         with open(json_file_path, 'r') as f:
16             data = json.load(f)
17
18         # Return the Python list as a JSON response using jsonify
19         return jsonify(data)
20     except FileNotFoundError:
21         return jsonify({"error": "Data file not found"}), 404
22     except json.JSONDecodeError:
23         return jsonify({"error": "Error decoding JSON from file"}), 500
24
25 if __name__ == '__main__':
26     # Run the application in debug mode
27     app.run(debug=True)
28
```

Data.json

```

{} data.json > ...
8      {
9          "id": 2,
10         "name": "Guna",
11         "Company": "Postiefts Technologies",
12         "laptop": "Lenovo"
13     },
14     {
15
16         "id": 3,
17         "name": "Koushek",
18         "Company": "Postiefts Technologies",
19         "laptop": "Asus"
20     },
21     {
22
23         "id": 4,
24         "name": "Santhosh",
25         "Company": "Postiefts Technologies",
26         "laptop": "Samsung"
27     }
28 ]

```

Explanation:

This Flask application provides an /api route that returns data in JSON format. The data is stored in a backend file (such as a data.json file). When the /api endpoint is accessed, the application reads the data from the file, converts it into a Python object, and sends it as a JSON response using Flask's jsonify function. This demonstrates how to build a simple API, read data from a file, and return it to the client in JSON format.

2. Create a form on the frontend that, when submitted, inserts data into MongoDB Atlas. Upon successful submission, the user should be redirected to another page displaying the message **"Data submitted successfully"**. If there's an error during submission, display the error on the same page without redirection.

App.py

```

app.py > ...
1 from flask import flask, render_template, request, redirect, url_for
2 from pymongo import MongoClient
3
4 app = Flask(__name__)
5
6 # Replace with your MongoDB Atlas connection string
7 MONGO_URI = "mongodb+srv://koushiksmenon5_db_user:Kailasml23@cluster0.1c3fzhs.mongodb.net/?appName=Cluster0"
8
9 client = MongoClient(MONGO_URI)
10 db = client["mydatabase"]
11 collection = db["users"]
12
13 @app.route('/')
14 def form():
15     return render_template('form.html', error=None)
16
17 @app.route('/submit', methods=['POST'])
18 def submit():
19     try:
20         name = request.form.get('name')
21         email = request.form.get('email')
22
23         if not name or not email:
24             return render_template('form.html', error="All fields are required")
25
26         # Insert into MongoDB
27         collection.insert_one({
28             "name": name,
29             "email": email
30         })
31
32         return redirect(url_for('success'))
33
34     except Exception as e:
35         return render_template('form.html', error=str(e))
36
37 @app.route('/success')
38 def success():
39     return render_template('success.html')
40
41 if __name__ == '__main__':
42     app.run(debug=True, use_reloader=False)

```

Form.html

```

templates > form.html > ...
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Form</title>
5 </head>
6 <body>
7     <h2>User Form</h2>
8
9     {% if error %}
10         <p style="color: red;">{{ error }}</p>
11     {% endif %}
12
13     <form action="/submit" method="POST">
14         <label>Name:</label><br>
15         <input type="text" name="name"><br><br>
16
17         <label>Email:</label><br>
18         <input type="email" name="email"><br><br>
19
20         <button type="submit">Submit</button>
21     </form>
22 </body>
23 </html>

```

Success.html

```
templates > <> success.html > ...
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Success</title>
5 </head>
6 <body>
7   <h2 style="color: green;">Data submitted successfully</h2>
8 </body>
9 </html>
```

Solution

User Form

Name:

Koushek

Email:

kousheksmenon8@gmail.o

Submit

127.0.0.1:5000/success

Data submitted successfully

ORGANIZATION
Itz me's Org - 2026-0...

PROJECT
Project 0

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

admin

local

mydatabase

users

sample_mflix

Cluster0 > mydatabase > users

Documents

Aggregations

Schema

Indexes

Validation

Type a query: { field: 'value' } or Generate query

Explain

Reset

Find

Options

25

1-1 of 1

_id: ObjectId('69c162233f7f9cfdde23535d')

name : "Koushek"

email : "kousheksmenon8@gmail.com"

Explanation:

This project is a Flask-based web application where users submit a form from the frontend. The entered data is sent to the backend and stored in MongoDB Atlas. If the submission is successful, the user is redirected to a success page showing "Data submitted successfully". If any error occurs, it is displayed on the same page without redirection. The application demonstrates basic form handling, database integration, and error management.